

REPÚBLICA DE ANGOLA

MINISTÉRIO DA EDUCAÇÃO

INSTITUTO TÉCNICO COMÉRCIAL DE LUANDA

DESENVOLVIMENTO DE UM SISTEMA WEB DE GESTÃO ESCOLAR PARA A AUTOMATIZAÇÃO DOS PROCESSOS ADMINISTRATIVOS E ACADÉMICOS

NOME: Virgilio Feliciano Jamba

Turma: CTI

N.º: 23

Luanda,Angola

2026

Resumo

O presente trabalho propõe o desenvolvimento de um sistema web de gestão escolar destinado a apoiar a organização dos processos administrativos e académicos. A proposta considera o registo de alunos, professores, turmas, matrículas, notas e assiduidade. O estudo apresenta um cenário hipotético, requisitos funcionais e não funcionais, análise de uma solução semelhante, tecnologias selecionadas, arquitetura em camadas e um modelo preliminar da base de dados. A solução proposta utiliza React, TypeScript, Next.js, Tailwind CSS, Node.js, PostgreSQL, Git e GitHub. Os benefícios esperados, como a centralização dos registos e a consulta mais consistente da informação, deverão ser confirmados através da implementação e de testes.

Palavras-chave: Gestão escolar; sistema web; automatização; base de dados; Informática de Gestão.

Sumário

1. Introdução e Enquadramento

1.1. Contextualização Histórica e Social

1.2. Justificação Científica e Prática do Tema

2. Estudo de Caso e Diagnóstico Institucional

2.1. Cenário de uma Instituição de Ensino

2.2. Fluxo Atual de Informação e Dificuldades Potenciais

3. Engenharia de Requisitos do Sistema

Levantamento de requisitos e fontes bibliográficas

3.1. Requisitos Funcionais (RF)

3.2. Requisitos Não Funcionais (RNF)

3.3. Utilizadores e Casos de Uso

4. Problematização e Questão de Investigação

4.1. Delimitação Clara do Problema Técnico

4.2. Questão de Investigação e Impactos Potenciais

5. Objetivos da Investigação

5.1. Objetivo Geral

5.2. Objetivos Específicos do Projeto

6. Estudo de Comparação e Estado da Arte

6.1. Análise de Soluções Existentes

6.2. Matriz Comparativa de Funcionalidades

7. Pilha Tecnológica Selecionada (Front-end)

7.1. React e TypeScript na Construção da Interface

7.2. Next.js e Tailwind CSS

8. Pilha Tecnológica Selecionada (Back-end e Dados)

8.1. Node.js e Organização da Lógica de Servidor

8.2. PostgreSQL e Modelo Relacional

8.3. Git e GitHub para Controlo de Versões
9. Arquitetura Estrutural e Fluxos de Dados
9.1. Modelo Cliente-Servidor em Camadas
9.2. Mecanismos de Validação e Segurança
10. Comparação Crítica das Ferramentas Computacionais
10.1. PostgreSQL e Alternativas Não Relacionais
10.2. Frameworks e Organização do Desenvolvimento
Síntese da comparação e justificação da seleção
11. Conclusões e Considerações Finais
11.1. Contribuições Principais da Proposta
11.2. Limitações Identificadas e Trabalhos Futuros
12. Referências Bibliográficas

1. Introdução e Enquadramento

1.1. Contextualização Histórica e Social

A utilização das tecnologias de informação tem alterado a forma como as organizações registam, conservam e consultam dados. Nas instituições de ensino, a informação sobre alunos, professores, turmas, matrículas, avaliações e assiduidade é necessária para o funcionamento administrativo e académico. Quando estes registos são mantidos em papel ou em ficheiros isolados, a consulta pode tornar-se demorada e a atualização dos dados pode exigir a repetição de tarefas.

Um sistema web de gestão escolar constitui uma possível resposta a essas necessidades, reunindo dados relacionados numa aplicação acessível através de um navegador. A proposta deste trabalho é académica e não pressupõe que uma escola específica tenha sido observada ou auditada. As dificuldades descritas são situações possíveis que deverão ser confirmadas através de um estudo de campo, caso o projeto venha a ser aplicado numa instituição real.

1.2. Justificação Científica e Prática do Tema

O tema justifica-se pela relação entre a Informática de Gestão e a organização dos processos de uma instituição de ensino. A nível prático, uma aplicação integrada pode apoiar o registo de alunos, a gestão de matrículas, a organização de turmas, o lançamento de notas e a consulta de relatórios. A nível académico, permite aplicar conhecimentos de análise de sistemas, programação, bases de dados, desenho de interfaces e segurança da informação.

A proposta não afirma que os benefícios já foram medidos. A redução de tarefas repetitivas, a rapidez de consulta e a melhoria da consistência dos dados são resultados esperados, que terão de ser avaliados após a implementação e a realização de testes.

2. Estudo de Caso e Diagnóstico Institucional

2.1. Cenário de uma Instituição de Ensino

Para efeitos deste trabalho, considera-se um cenário hipotético de uma instituição de ensino que necessita de organizar informações de alunos, professores, turmas, disciplinas, matrículas, notas e assiduidade. Não foi fornecido o nome de uma escola real nem foram apresentados dados

recolhidos no terreno; por isso, este cenário serve para orientar a análise e não deve ser interpretado como diagnóstico comprovado de uma instituição específica.

O sistema proposto deverá permitir que cada utilizador aceda apenas às funções adequadas ao seu papel. Por exemplo, a secretaria poderá gerir matrículas, os professores poderão consultar as turmas atribuídas e registar avaliações, enquanto a direção poderá consultar relatórios gerais de acordo com as permissões definidas.

2.2. Fluxo Atual de Informação e Dificuldades Potenciais

Num processo não integrado, uma informação pode ser registada em documentos diferentes e depois transcrita para folhas de cálculo ou outros ficheiros. Este fluxo pode originar duplicação, divergências entre registos e demora na preparação de mapas. O projeto parte destas possibilidades como hipóteses a investigar.

Fluxo de trabalho hipotético: receção de dados → registo manual ou em ficheiros separados → conferência → procura de informação → elaboração de mapas. No sistema proposto, o fluxo será: autenticação → seleção da função → validação dos dados → gravação centralizada → consulta ou emissão de relatório.

A confirmação das dificuldades reais exigiria entrevistas com os responsáveis, observação dos procedimentos e análise dos documentos utilizados pela escola escolhida.

3. Engenharia de Requisitos do Sistema

Levantamento de requisitos e fontes bibliográficas

O levantamento de requisitos identifica o que os utilizadores precisam que o sistema faça e as condições de qualidade que deve cumprir. Os requisitos apresentados neste trabalho são preliminares e deverão ser confirmados com a direção, a secretaria, os professores e o responsável informático. Recomenda-se realizar entrevistas, observar os procedimentos de matrícula e avaliação, analisar fichas e mapas usados pela escola e validar os resultados com os futuros utilizadores.

O levantamento bibliográfico deve incluir documentação oficial das tecnologias selecionadas, referências sobre engenharia de requisitos, orientações de segurança e documentação de sistemas escolares semelhantes. Essas fontes fundamentam as decisões técnicas, mas não substituem a

confirmação das necessidades reais da instituição. Devem ser registadas as fontes efetivamente consultadas e as datas de acesso exigidas pela escola.

3.1. Requisitos Funcionais (RF)

RF01 — O sistema deve permitir a autenticação dos utilizadores autorizados.

RF02 — Deve permitir criar, consultar, atualizar e desativar registos de alunos.

RF03 — Deve permitir gerir professores, turmas e disciplinas.

RF04 — Deve permitir registar e consultar matrículas.

RF05 — Deve permitir associar alunos a turmas e anos letivos.

RF06 — Deve permitir registar notas por disciplina e período, de acordo com as permissões.

RF07 — Deve permitir registar e consultar a assiduidade.

RF08 — Deve permitir pesquisar registos através de critérios definidos.

RF09 — Deve permitir gerar relatórios de gestão a partir dos dados existentes.

RF10 — Deve permitir administrar os perfis e as permissões dos utilizadores.

3.2. Requisitos Não Funcionais (RNF)

RNF01 — Usabilidade: as páginas devem apresentar navegação clara e linguagem compreensível.

RNF02 — Segurança: o acesso deve ser autenticado e controlado por permissões.

RNF03 — Integridade: os dados devem obedecer a validações e relações definidas na base de dados.

RNF04 — Desempenho: as operações comuns devem responder em tempo adequado ao ambiente de instalação.

RNF05 — Compatibilidade: a interface deve funcionar em navegadores modernos e adaptar-se a ecrãs de diferentes dimensões.

RNF06 — Manutenção: o código deve ser organizado em módulos e acompanhado por controlo de versões.

RNF07 — Recuperação: devem ser definidos procedimentos de cópia de segurança e restauro.

RNF08 — Privacidade: os dados pessoais e académicos devem ser acessíveis apenas a utilizadores autorizados.

3.3. Utilizadores e Casos de Uso

Os perfis inicialmente considerados são Administrador, Direção, Secretaria e Professor. O Administrador configura contas e permissões; a Direção consulta indicadores e relatórios autorizados; a Secretaria gere dados administrativos; o Professor consulta as turmas atribuídas e regista avaliações e assiduidade quando autorizado.

Caso de uso principal: o utilizador inicia sessão, o sistema verifica as credenciais e o perfil, apresenta as opções permitidas, valida a operação solicitada e regista a alteração quando os dados forem válidos. As permissões finais devem ser confirmadas com a instituição que venha a utilizar o sistema.

4. Problematização e Questão

4.1. Delimitação Clara do Problema Técnico de Investigação

O problema técnico abordado é a necessidade de organizar, relacionar e proteger os dados administrativos e académicos numa aplicação integrada. A proposta concentra-se nos módulos de alunos, professores, turmas, disciplinas, matrículas, notas, assiduidade e relatórios. Não inclui, nesta fase, processamento de pagamentos, ensino à distância ou integração automática com plataformas governamentais, pois essas funções exigiriam requisitos adicionais.

4.2. Questão de Investigação e Impactos Potenciais

Questão de investigação: de que forma o desenvolvimento de um sistema web de gestão escolar pode apoiar a organização e a consulta dos processos administrativos e académicos de uma instituição de ensino?

Entre os impactos potenciais encontram-se a centralização dos registos, a redução de duplicações, a consulta mais consistente da informação e o apoio à elaboração de relatórios. Estes impactos são expectativas do projeto e não resultados comprovados. A sua avaliação deverá considerar indicadores como o tempo necessário para localizar um registo, a quantidade de erros encontrados em testes e a percentagem de operações concluídas corretamente.

5. Objetivos da Investigação

5.1. Objetivo Geral

Desenvolver um sistema web de gestão escolar para a automatização dos processos administrativos e académicos de uma instituição de ensino, integrando o registo e a consulta de alunos, professores, turmas, matrículas, notas e assiduidade.

5.2. Objetivos Específicos do Projeto

- Identificar os processos escolares que poderão beneficiar da informatização.
- Analisar um sistema semelhante e reconhecer funcionalidades úteis para a proposta.
- Definir os requisitos funcionais e não funcionais.
- Selecionar tecnologias adequadas à interface, à lógica da aplicação e à persistência de dados.
- Propor uma arquitetura e um modelo preliminar da base de dados.
- Definir medidas de segurança, protótipos de interface e testes de validação.
- Indicar critérios para avaliar o funcionamento do sistema após a implementação.

6. Estudo de Comparação e Estado da Arte

6.1. Análise de Soluções Existentes

A análise de soluções semelhantes ajuda a reconhecer funcionalidades recorrentes em sistemas de gestão escolar. Como referência documental, pode ser estudado o Frappe Education, um projeto de software para gestão educacional associado ao ecossistema Frappe/ERPNext. A consulta da documentação oficial deve ser usada para confirmar as funcionalidades disponíveis na versão analisada; não se presume que todas as funcionalidades desse produto sejam reproduzidas neste projeto.

A proposta própria tem âmbito académico e foca-se nos módulos definidos nos requisitos. O estudo comparativo deve observar a gestão de estudantes, turmas, matrículas, avaliações, relatórios, controlo de acessos, facilidade de utilização e possibilidades de adaptação.

6.2. Matriz Comparativa de Funcionalidades

Critérios de comparação propostos:

Solução de referência — Frappe Education: sistema existente, com funcionalidades educacionais documentadas pelo projeto; a avaliação concreta deve ser feita com base na documentação e demonstração da versão consultada.

Sistema proposto — Gestão Escolar Web: solução a desenvolver, com âmbito definido neste trabalho e tecnologias selecionadas para fins académicos.

Matriz de análise:

- Gestão de alunos e matrículas — verificar na solução de referência; previsto na proposta.
- Turmas e disciplinas — verificar na solução de referência; previsto na proposta.
- Notas e assiduidade — verificar na solução de referência; previsto na proposta.
- Perfis e permissões — examinar a documentação; previsto como requisito de segurança.
- Relatórios — verificar exemplos existentes; previstos para consulta de dados.
- Personalização — analisar as opções documentadas; dependerá da implementação da proposta.

Esta matriz indica o que deve ser comparado e não apresenta pontuações de desempenho, pois não foram realizados testes práticos equivalentes entre os sistemas.

7. Pilha Tecnológica Selecionada (Front-end)

7.1. React e TypeScript na Construção da Interface

React será utilizado para construir a interface a partir de componentes reutilizáveis, como formulários, tabelas, menus e painéis de resumo. Esta organização facilita a separação das funcionalidades e a manutenção da interface. TypeScript acrescenta tipagem estática ao JavaScript e pode ajudar a detetar alguns erros durante o desenvolvimento.

A interface deverá incluir páginas para autenticação, painel inicial, alunos, professores, turmas, matrículas, notas, assiduidade e relatórios. Os formulários devem apresentar mensagens de validação e confirmar ações importantes antes de alterações destrutivas.

7.2. Next.js e Tailwind CSS

Next.js será considerado como estrutura de aplicação React, apoiando a organização das páginas e a integração das partes da aplicação. Tailwind CSS será utilizado para compor os estilos através de classes utilitárias e manter uma apresentação consistente. A escolha final das

versões e da configuração deve ser feita no início da implementação, consultando a documentação oficial correspondente.

O desenho visual deve priorizar legibilidade, contraste adequado, navegação simples e adaptação a computadores e dispositivos móveis. O protótipo apresentado no anexo é ilustrativo; não representa uma aplicação já implementada.

8. Pilha Tecnológica Selecionada (Back-end e Dados)

8.1. Node.js e Organização da Lógica de Servidor

Node.js será utilizado no ambiente de execução do servidor. A camada de servidor deverá receber pedidos, validar os dados, verificar as permissões do utilizador e encaminhar operações para a base de dados. A lógica de negócio não deve depender apenas da validação feita no navegador, porque os pedidos podem ser enviados diretamente ao servidor.

A aplicação deverá tratar erros de forma controlada, evitar a exposição de credenciais e registar eventos relevantes para manutenção, sem guardar palavras-passe em texto simples.

8.2. PostgreSQL e Modelo Relacional

PostgreSQL será utilizado para armazenar os dados de forma estruturada. O modelo preliminar considera as entidades Utilizador, Aluno, Professor, Turma, Disciplina, Matrícula, Nota e Assiduidade. As chaves primárias identificam os registos e as chaves estrangeiras estabelecem relações entre entidades.

Modelo conceptual simplificado: ALUNO 1 — N MATRÍCULA N — 1 TURMA TURMA 1 — N DISCIPLINA (a associação definitiva depende da organização curricular) ALUNO 1 — N NOTA N — 1 DISCIPLINA ALUNO 1 — N ASSIDUIDADE UTILIZADOR 1 — N REGISTOS DE AUDITORIA

A estrutura final deverá definir campos, tipos, restrições, regras para evitar duplicação e tratamento de alterações de ano letivo. A relação entre turmas e disciplinas poderá exigir uma tabela de associação própria, conforme os requisitos reais.

8.3. Git e GitHub para Controlo de Versões

Git será usado para acompanhar as alterações ao código, enquanto GitHub poderá alojar o repositório remoto e facilitar a colaboração. Recomenda-se criar commits pequenos e descritivos, manter uma versão principal estável e documentar as instruções de instalação. O repositório não deve conter palavras-passe, chaves privadas nem dados pessoais reais de alunos.

9. Arquitetura Estrutural e Fluxos de Dados

9.1. Modelo Cliente-Servidor em Camadas

A arquitetura proposta separa a interface, a lógica de aplicação e a persistência de dados. O navegador apresenta a interface; a aplicação recebe e valida os pedidos; a base de dados armazena os registos. Esta separação ajuda a organizar responsabilidades e permite testar cada parte com maior clareza.

Fluxo principal: [Utilizador / Navegador] → [Interface React / Next.js] → [Servidor Node.js: autenticação, validação e regras] → [PostgreSQL] A resposta percorre o caminho inverso e apresenta apenas os dados que o utilizador tem autorização para consultar.

Funcionamento conjunto das tecnologias: o navegador apresenta a interface criada com React, organizada por Next.js e estilizada com Tailwind CSS. TypeScript ajuda a definir os tipos dos dados usados no código. Quando o utilizador envia um formulário, a interface envia um pedido ao servidor Node.js. O servidor verifica a autenticação e as permissões, valida os campos e aplica as regras de negócio; depois consulta ou grava os dados no PostgreSQL. A base de dados aplica as relações e restrições definidas, e o servidor devolve uma resposta que a interface apresenta ao utilizador. Git regista as alterações realizadas durante o desenvolvimento e GitHub pode alojar o repositório remoto; estas duas ferramentas apoiam os programadores e não processam diretamente as matrículas ou notas durante a utilização normal.

9.2. Mecanismos de Validação e Segurança

A segurança deverá incluir autenticação, autorização por perfil, validação no servidor, consultas parametrizadas, gestão segura de sessões, proteção de credenciais, cópias de segurança e atualização das dependências. As palavras-passe devem ser armazenadas com um algoritmo de hash apropriado, nunca em texto simples. As permissões devem ser verificadas em cada operação protegida.

Os testes deverão incluir tentativas de acesso sem sessão, acesso a funções de outro perfil, submissão de campos inválidos, duplicação de matrículas e consulta de dados não autorizados. A implementação deve seguir orientações de segurança reconhecidas, como as recomendações OWASP, adaptadas à arquitetura escolhida.

10. Comparação Crítica das Ferramentas Computacionais

10.1. PostgreSQL e Alternativas Não Relacionais

PostgreSQL é uma base de dados relacional adequada a dados com relações explícitas e regras de integridade, como matrículas, alunos, turmas e notas. Uma solução NoSQL pode ser apropriada para outros tipos de dados e padrões de acesso, mas a escolha deve depender dos requisitos, não apenas da popularidade da tecnologia. Para esta proposta, o modelo relacional é coerente com a necessidade de manter relações consistentes entre entidades escolares.

10.2. Frameworks e Organização do Desenvolvimento

A combinação de React, Next.js, TypeScript e Tailwind CSS organiza a construção da interface, enquanto Node.js e PostgreSQL cobrem a lógica de servidor e a persistência. A vantagem esperada é a divisão clara de responsabilidades e a possibilidade de manutenção modular. Em contrapartida, a equipa terá de aprender e configurar várias tecnologias, gerir dependências e manter compatibilidade entre versões.

A escolha deve ser reavaliada se os requisitos mudarem, se a infraestrutura disponível for limitada ou se a instituição já possuir sistemas que exijam integração. Nenhuma ferramenta, isoladamente, garante segurança, desempenho ou qualidade: estes aspetos dependem também da arquitetura, da implementação, dos testes e da manutenção.

Síntese da comparação e justificação da seleção

Para a interface, React foi selecionado em vez de alternativas como Vue ou Angular por permitir organizar a aplicação em componentes reutilizáveis; TypeScript foi escolhido em vez de JavaScript sem tipagem estática para ajudar a detetar inconsistências no código; Next.js complementa React com convenções de estrutura da aplicação; e Tailwind CSS foi escolhido em vez de CSS escrito integralmente de forma manual para uniformizar estilos. Estas escolhas favorecem a organização, mas exigem aprendizagem e configuração.

Para o servidor, Node.js foi escolhido em vez de alternativas como PHP, Python ou Java por permitir manter o desenvolvimento no ecossistema JavaScript/TypeScript. Para os dados, PostgreSQL foi escolhido em vez de uma solução NoSQL porque os registos escolares possuem relações claras entre alunos, matrículas, turmas e notas. Git acompanha as versões do código e GitHub pode alojar o repositório remoto. A decisão deve considerar os requisitos confirmados, a experiência da equipa, a infraestrutura e a manutenção; nenhuma ferramenta garante, isoladamente, qualidade ou segurança.

11. Conclusões e Considerações Finais

11.1. Contribuições Principais da Proposta

O trabalho organizou uma proposta de sistema web de gestão escolar a partir de um cenário hipotético, identificou requisitos funcionais e não funcionais, definiu perfis de utilizadores e selecionou tecnologias para a interface, o servidor, a base de dados e o controlo de versões. Também apresentou uma arquitetura em camadas, um modelo relacional preliminar e medidas de segurança a considerar na implementação.

Conclui-se que uma aplicação integrada poderá apoiar a organização e a consulta dos registos escolares, desde que seja implementada de acordo com os requisitos e validada com utilizadores reais. Como não foi construída e testada uma versão funcional no âmbito deste documento, os benefícios descritos permanecem resultados esperados.

11.2. Limitações Identificadas e Trabalhos Futuros

A principal limitação é a ausência de recolha de dados numa instituição concreta e de testes de uma aplicação implementada. Por isso, o estudo não apresenta estatísticas locais nem declara que os requisitos foram validados por uma escola. Como trabalho futuro, recomenda-se selecionar uma instituição, entrevistar os responsáveis, confirmar os processos existentes, elaborar protótipos com os utilizadores, implementar um módulo inicial e realizar testes de aceitação.

Após a implementação, poderão ser avaliados o tempo de execução das tarefas, a ocorrência de erros, a facilidade de utilização, a segurança dos acessos e a qualidade dos relatórios. A integração de pagamentos, comunicação com encarregados de educação ou plataformas externas só deverá ser acrescentada após análise de requisitos própria.

12. Referências Bibliográficas

Referências consultáveis Frappe Technologies. Frappe Education — documentação e projeto oficial. https://frappe.io/education

Meta Open Source. React — documentação oficial. https://react.dev/

Microsoft. TypeScript — documentação oficial. https://www.typescriptlang.org/docs/

OpenJS Foundation. Node.js — documentação oficial. https://nodejs.org/docs/

Vercel. Next.js — documentação oficial. https://nextjs.org/docs

PostgreSQL Global Development Group. PostgreSQL — documentação oficial. https://www.postgresql.org/docs/

Tailwind Labs. Tailwind CSS — documentação oficial. https://tailwindcss.com/docs

Git. Git — documentação oficial. https://git-scm.com/doc

GitHub. GitHub Docs. https://docs.github.com/

OWASP Foundation. Cheat Sheet Series: Authentication, Authorization e Password Storage. https://cheatsheetseries.owasp.org/